COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/26/2026	**DATE SUBMITTED:**	07/28/2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS ON ACTIVITY NO. 3.1 Linked Lists

1. Objective(s)

- Implement the list ADT using singly and doubly linked lists
- Define operations based on list ADT from the module discussion

2. Intended Learning Outcomes (ILOs)

After this activity, the student should be able to:

a. Construct C++ code for a singly and doubly linked list in C++
b. Solve given problems utilizing linked lists in C++

3. Discussion

PART A: What is a linked list?

Linked Lists are a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers.

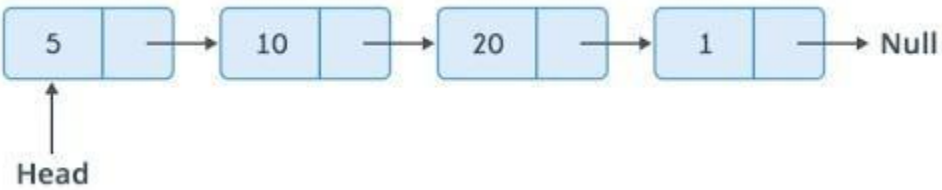


Image Source: Hackearth.com

Why Linked List?

- Arrays are useful but have the following limitations:
- Fixed size.
- Allocated memory remains to be the upper limit.
- Expensive operations (such as insertion) which requires movement of all existing elements and creation of room for new elements.

Advantages over Arrays

- Dynamic size

- Ease of insertion/deletion

Drawbacks

- Random access is not allowed. We have to access elements sequentially starting from the first node
- Extra memory space for a pointer is required with each element of the list.
- Not cache friendly. No locality reference.

Representation:

- A linked list is represented by a pointer to the first node of the linked list. The first node is called the head. If the linked list is empty, then the value of the head is NULL.
- Each node in a list consists of at least two parts:
    - Data
    - Pointer (Or Reference) to the next node

PART B: Doubly Linked Lists

Doubly Linked Lists are traversed in either direction. It is a linked list in which every node has a next pointer and a backpointer.

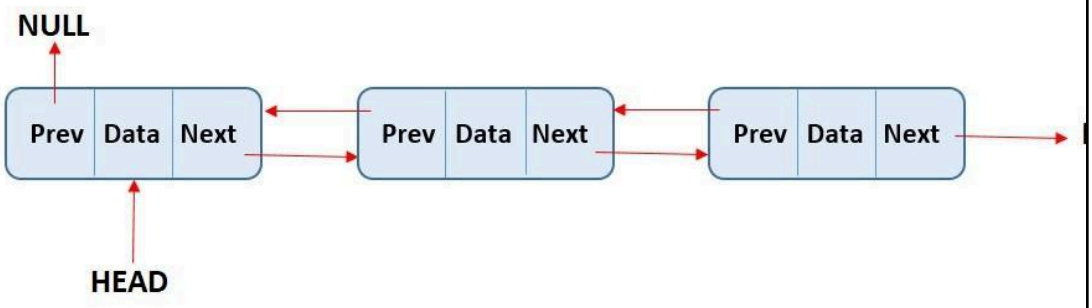


Image Source: AlphaCodingSkills

Every node contains address of next node (except the last node). Every node contains address of previous node (except the first node).

PART C: Common Operations

on Linked Lists Typical

operations:

- Initialize the list
- Destroy the list
- Determine if list empty
- Search list for a given item
- Insert an item
- Delete an item, and so on

4. Materials and Equipment

Personal Computer with C++ IDE
Recommended IDE:

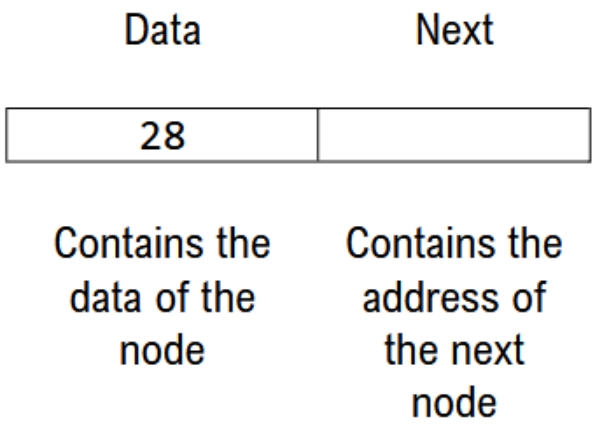
- CLion (must use TIP email to download)
- DevC++ (use the embarcadero fork or configure to C++17)

5. PROCEDURE

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

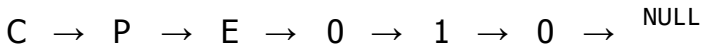
To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list:



Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list. In C++, the node is defined as:

```
class Node{
public:
    char data;
    Node *next;
};
```

We will implement a list to represent the string “CPE010” that looks like the figure below:



Implementation will follow the given steps:

1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null Simple implementation:

```
#include <iostream>
#include <utility>

class Node {
public:
    char data;
    Node* next;
};

int main() {
    // Step 1
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;
    Node* fourth = NULL;
    Node* fifth = NULL;
    Node* last = NULL;

    // Step 2
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;

    // Step 3
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;
```

```
        third->data = 'E';
        third->next = fourth;

        fourth->data = '0';
        fourth->next = fifth;

        fifth->data = '1';
        fifth->next = last;

        // Step 4
        last->data = '0';
        last->next = nullptr;

        return 0;
    }
```

Although we have created the linked list, this is an implementation that is useless and meant only to show what we want to happen for the given output. **Run your code and screenshot the output, then briefly discuss how the output came to be and how could it be improved (table 3-1)?**

Imagine if we had to make multiple items in the list in the hundreds or thousands! This would be too tedious, ineffective, and inefficient. So, we must implement certain methods. These methods/operations associated with the linked lists are:

- Traversal
- Insertion at head
- Insertion at any part of the list
- Insertion at the end
- Deletion of a node Linked List Traversal

```
Algorithm: ListTraversal (parameter: pointer to node n)
WHILE n IS NOT EQUAL TO null
      PRINT data OF n
      GO TO NEXT NODE n := next
      ENDWHILE
PRINT
next
line
END
```

Sometimes we have a linked list, and we need to insert a node somewhere other than at the end of the list. We will look at a couple of different ways to insert a node into an existing list.

To insert a node at the head:

1. Allocate memory for the new node
2. Put our data into the new node
3. Set Next of the new node to point to the previous Head
4. Reset Head to point to the new node

To insert a node at any location between the head and tail:

1. Check if it is the head node (previous node is null)
2. If null, print "Previous node cannot be null."
3. Allocate a new node
4. Store data in the new node
5. Point new node to the node previous node was pointing to
6. Point previous node to the new node

To insert a node at the end:

1. Allocate new node
2. Dereference to the head node
3. Store data in new node
4. Point next of new node to NULL
5. Traverse the list until next of the node is null
6. Point the next of the current node to the new node

To delete a node from linked list:

1. Find previous node of the node to be deleted.
2. Change the next of previous node.
3. Free memory for the node to be deleted.

Create code for each of the pseudocode given for all list operations above. Provide screenshots in table 3-2.

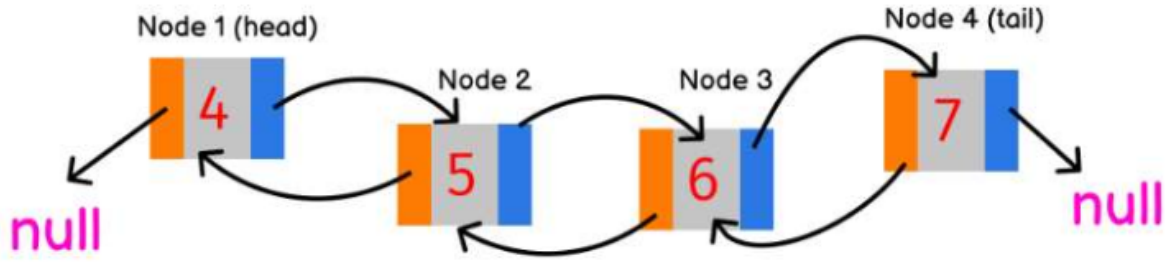
In your driver function, show the use of each list operation and show the output in table 3-3 found in section 6 with a descriptive caption for each. The tasks you have to perform are as follows:

a. Traverse the list by passing the head of the created list into the function
b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"
c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".
d. Delete the node containing the element C.
e. Delete the node containing the element P.

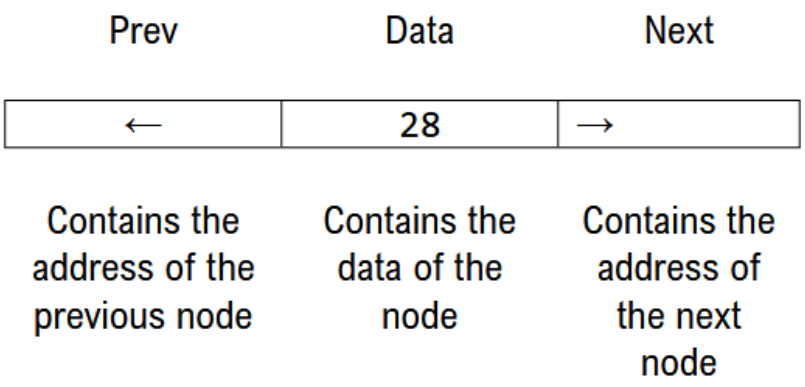
f. Show the elements in the list. Output should be “GEE101”.

A.2. Doubly Linked List

The singly linked list allows for direct access from a list node only to the next node in the list. A doubly linked list allows convenient access from a list node to the next node and also to the preceding node on the list.



The doubly linked list node accomplishes this in the obvious way by storing two pointers: one to the node following it (as in the singly linked list), and a second pointer to the node preceding it.



This means that in addition to our implementation of the singly linked list, we’ll have addition compartment.

```
class Node{
public:
char data;
Node *next;
Node *prev;
};
```

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4

6. OUTPUT

Screenshot

```cpp
#ifndef SIMPLY_LL_H
#define SIMPLY_LL_H

template <typename T>
class SimplyList {
public:
    T data;
    SimplyList<T> *next = nullptr;
};

template <typename T>
void listTraversal(SimplyList<T>* head) {
    if (head == nullptr) {
        std::cout << "List is Empty\n";
        return;
    }

    while (head != nullptr) {
        std::cout << head->data;
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
}

#endif
```

HOA3.1_JDC > main.cpp > main()

```cpp
#include <iostream>
#include "HOA3.1_JDC_singly_LL.h"

int main() {
    SimplyList<char>* head = new SimplyList<char>();
    SimplyList<char>* second = new SimplyList<char>();
    SimplyList<char>* third = new SimplyList<char>();
    SimplyList<char>* fourth = new SimplyList<char>();
    SimplyList<char>* fifth = new SimplyList<char>();
    SimplyList<char>* last = new SimplyList<char>();


    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '0';
    fourth->next = fifth;

    fifth->data = '1';
    fifth->next = last;

    last->data = '0';
    last->next = nullptr;

    listTraversal(head);
    std::cout << std::endl;

    SimplyList<char>* current = head;
    while (current != nullptr) {
        SimplyList<char>* nextNode = current->next;
        delete current;
        current = nextNode;
    }

    return 0;
}
```

	
Discussion	The Node class only uses char, there are no reusable functions for adding, removing, or navigating the list, everything is hardcoded into a single main() function, and it combines nullptr and NULL. It is better to introduce a header since it keeps the data structure and its use apart. Main.cpp merely calls the SimplyList class and its operations, which are contained in a header. Since it is a template, it uses nullptr consistently and can handle any data type rather than just char.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

Insertion At Head	```
	template <typename T>
	void sllinsertHead(T newData, SimplyList<T>** currentHead) {
	    SimplyList<T>* newNode = new SimplyList<T>;
	
	    newNode->data = newData;
	
	    newNode->next = *currentHead;
	
	    *currentHead = newNode;
	}
	``
	Inserting a node at the head: G->J->A->R->E->D

Insertion at any part of the list	```
	template <typename T>
	void sllGeneralInsert(T newData, SimplyList<T>* prevNode) {
	    if (prevNode == nullptr) {
	        std::cout << "Previous Node Cannot Be Null\n";
	        return;
	    }
	    SimplyList<T>* newNode = new SimplyList<T>;
	
	    newNode->data = newData;
	    newNode->next = prevNode->next;
	
	    prevNode->next = newNode;
	}
	``
	Inserting a new node at the third section: G->J->A->W->R->E->D

Insertion at the end	```
	template <typename T>
	void sllInsertEnd(T newData, SimplyList<T>** head) {
	    SimplyList<T>* newNode = new SimplyList<T>;
	    SimplyList<T>* currentNode = *head;
	    newNode->data = newData;
	    newNode->next = nullptr;
	
	    while (currentNode->next != nullptr) {
	        currentNode = currentNode->next;
	    }
	    currentNode->next = newNode;
	}
	``
	Inserting a new node at the end: G->J->A->X->R->E->D->K

Deletion of node	

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head) {
    if (*head == nullptr) return;

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if (currNode == nullptr) return;

    if (prevNode == nullptr) {
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```
Deleting a node: G->J->A->R->E->D->Z
```

Table 3-2. Code for the List Operations

a.	**Source Code**	

```
template <typename T>
void listTraversal(SimplyList<T>* head) {
    if (head == nullptr) {
        std::cout << "List is Empty\n";
        return;
    }

    while (head != nullptr) {
        std::cout << head->data;
        if (head->next != nullptr) {
            std::cout << "->";
        }
        head = head->next;
    }
}
```

		``
		std::cout << "Original List: ";
		listTraversal(head);
		std::cout << std::endl;
		``
		In order to safely print "List is Empty" rather than crashing on a null list, listTraversal(head) checks for head == nullptr up front. Additionally, because head is passed by value, moving it forward inside the loop doesn't affect the caller's original pointer. listTraversal(head) walks through the list starting at head, printing each node's data followed by "->" if there is a next node, and stops adding the arrow after the last node.

	Screenshot	

b.	**Source Code**	```
		template <typename T>
		void sllinsertHead(T newData, SimplyList<T>** currentHead) {
		    SimplyList<T>* newNode = new SimplyList<T>;
		    newNode->data = newData;
		    newNode->next = *currentHead;
		    *currentHead = newNode;
		}
		
		sllinsertHead('G', &head);
		std::cout << "Inserting a node at the head: ";
		listTraversal(head);
		std::cout << std::endl;
		``
		sllinsertHead('G', &head) places a new node containing 'G' at the very front of the list; because it takes &head, it can reassign what head points to, so after this call the list becomes G->C->P->E->1->0->1, and listTraversal(head) prints that updated result. It creates the new node, links its next to the old head, then reassigns *currentHead so the caller's head variable now points to the new node.

	Screenshot	

c.

Source Code

```
template <typename T>
void sllGeneralInsert(T newData, SimplyList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous Node Cannot Be Null\n";
        return;
    }
    SimplyList<T>* newNode = new SimplyList<T>;

    newNode->data = newData;
    newNode->next = prevNode->next;

    prevNode->next = newNode;
}
```

```
sllGeneralInsert('X', second);
std::cout << "Inserting a node at the fourth: ";
listTraversal(head);
std::cout << std::endl;
```

The function sllGeneralInsert('E', head->next->next) inserts a new node with 'X' right after the node that head->next->next is currently referring to. The list becomes G->C->P->X->E->1->0->1 after it first verifies that prevNode isn't nullptr. At this point, head is G, so head->next is C and head->next->next is P.

Screenshot

```
Inserting a node at the fourth: G->C->P->X->E->1->0->1
```

d.

Source Code

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head) {
    if (*head == nullptr) return;

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if (currNode == nullptr) return;

    if (prevNode == nullptr) {
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```
sllDeleteNode('X', &head);
std::cout << "Deleting a node: ";
listTraversal(head);
std::cout << std::endl;
```

The first sllDeleteNode('C', &head) searches the list for a node whose data equals 'C', finds it as the second node, and removes it by relinking G's next pointer directly to P; since 'C' is not the head node in this call, it takes the prevNode->next = currNode->next branch inside the function, and the list becomes G->P->E->E->1->0->1.

Screenshot

```
Deleting a node: G->C->P->E->1->0->1->Z
```

e. **Source Code**

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head) {
    if (*head == nullptr) return;

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if (currNode == nullptr) return;

    if (prevNode == nullptr) {
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```
sllDeleteNode('Z', &head);
std::cout << "Deleting a node: ";
listTraversal(head);
std::cout << std::endl;
```

The process for this is the same as the first sllDeleteNode function but adds 'Z' as the variable to delete.

Screenshot

```
Deleting a node: G->C->P->E->1->0->1
```

f. **Source Code**

```
1   #ifndef SIMPLY_LL_H
2   #define SIMPLY_LL_H
3
4   #include <iostream>
5
6   template <typename T>
7   class SimplyList {
8   public:
9       T data;
10      SimplyList<T> *next = nullptr;
11  };
```

```
int main() {
    SimplyList<char>* head = new SimplyList<char>();
    SimplyList<char>* second = new SimplyList<char>();
    SimplyList<char>* third = new SimplyList<char>();
    SimplyList<char>* fourth = new SimplyList<char>();
    SimplyList<char>* fifth = new SimplyList<char>();
    SimplyList<char>* last = new SimplyList<char>();

    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '1';
    fourth->next = fifth;

    fifth->data = '0';
    fifth->next = last;

    last->data = '1';
    last->next = nullptr;
```

```
std::cout << "Original List: ";
listTraversal(head);
std::cout << std::endl;

sllinsertHead('G', &head);
std::cout << "Inserting a node at the head: ";
listTraversal(head);
std::cout << std::endl;

sllGeneralInsert('X', second);
std::cout << "Inserting a node at the fourth: ";
listTraversal(head);
std::cout << std::endl;

sllInsertEnd('Z', &head);
std::cout << "Inserting a node at the end: ";
listTraversal(head);
std::cout << std::endl;

sllDeleteNode('X', &head);
std::cout << "Deleting a node: ";
listTraversal(head);
std::cout << std::endl;

sllDeleteNode('Z', &head);
std::cout << "Deleting a node: ";
listTraversal(head);
std::cout << std::endl;
```

Screenshot

```
Original List: C->P->E->1->0->1
Inserting a node at the head: G->C->P->E->1->0->1
Inserting a node at the fourth: G->C->P->X->E->1->0->1
Inserting a node at the end: G->C->P->X->E->1->0->1->Z
Deleting a node: G->C->P->E->1->0->1->Z
Deleting a node: G->C->P->E->1->0->1
```

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis

``
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H
 	
#include <iostream>
 	
template <typename T>
class DoublyList {
public:
    T data;
    DoublyList<T>* prev;
    DoublyList<T>* next;
};
``
	template class DoubleList specifies the structure of the node. Each node contains a data field of the generic type T, along with two pointers: prev, which refers to the preceding node, and next, which refers to the subsequent node. This is what renders it "doubly" linked, in contrast to a singly-linked list that contains only next; the additional prev pointer facilitates backward movement and allows you to delete or insert a node without traversing the list from the head to locate its predecessor.

``
template <typename T>
DoublyList<T>* createNode(T newData) {
    DoublyList<T>* newNode = new DoublyList<T>;
    newNode->data = newData;
    newNode->prev = nullptr;
    newNode->next = nullptr;
    return newNode;
}
``
	template DoubleList* createNode(T newData) is a concise factory function that allocates a new node using new DoubleList, assigns its data to newData, and sets both prev and next to nullptr because a newly created node isn't connected to any other node at this point. It provides the pointer to this new node, allowing the caller to insert it into a list.

``
template <typename T>
void listTraversal(DoublyList<T>* head) {
    if (head == nullptr) {
        std::cout << "List is Empty\n";
        return;
    }
 	
    DoublyList<T>* tail = nullptr;
    DoublyList<T>* current = head;
 	
    std::cout << "Forward: ";
    while (current != nullptr) {
        std::cout << current->data;
        if (current->next != nullptr) {
            std::cout << "<->";
        }
        tail = current;
        current = current->next;
    }
    std::cout << std::endl;
``
	template void dllTraverse(DoubleList* currentNode) displays the list in both ways. It initially verifies whether the list is empty and returns immediately if that is the case (once corrected). Alternatively, it traverses forward from currentNode to the end, displaying each node's data and storing the last visited node in tail; afterward, it moves backward from tail using prev pointers back to the head, printing once more — showcasing the list's bidirectional characteristic.

``
std::cout << "Backward: ";
while (tail != nullptr) {
    std::cout << tail->data;
    if (tail->prev != nullptr) {
        std::cout << "<->";
    }
    tail = tail->prev;
}
std::cout << std::endl;
``	

``
template <typename T>
void dllInsertHead(T newData, DoublyList<T>** currentHead) {
    DoublyList<T>* newNode = createNode(newData);
    newNode->next = *currentHead;
 	
    if (*currentHead != nullptr) {
        (*currentHead)->prev = newNode;
    }
 	
    *currentHead = newNode;
}
``
	template void dllInsertHead(T newData, DoubleList** currentHead) adds a new node to the beginning of the list. It requires a pointer-to-pointer (DoubleList**) since it needs to modify what the head variable of the caller references. It generates the node, connects newNode->next to the previous head, links the previous head's prev back to newNode, and then updates *currentHead to the new node. Be aware this version requires a null-check on the old head before dereferencing (*currentHead)->prev, because if the list begins empty, *currentHead is nullptr, leading to a crash.

``
template <typename T>
void dllInsertEnd(T newData, DoublyList<T>* currentHead) {
    DoublyList<T>* newNode = createNode(newData);
 	
    while (currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }
 	
    currentHead->next = newNode;
    newNode->prev = currentHead;
}
``
	template void dllInsertEnd(T newData, DoubleList* currentHead) adds a new node at the tail of the list. It requires just one pointer, rather than a double pointer, because adding to the end does not alter the position of head. It advances currentHead until currentHead->next == nullptr (the final node), creates a new node, connects the last node's next to it, and connects the new node's prev back to the last node — finalizing the bidirectional link at the tail.

```cpp
template <typename T>
void dllInsertAny(T newData, DoublyList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous Node Cannot Be Null\n";
        return;
    }

    DoublyList<T>* newNode = createNode(newData);
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != nullptr) {
        prevNode->next->prev = newNode;
    }

    prevNode->next = newNode;
}
```

template void dllInsertAny(...) (or dllGeneralInsert, once corrected) places a new node following a specified node at any location in the list. A single pointer to the node after which you want to insert (prevNode) is required, it verifies that prevNode is not nullptr, creates the new node, assigns its next to prevNode's old next and its prev to prevNode, then — if there was a node after prevNode — adjusts that node's prev to link back to the new node, and ultimately updates prevNode->next to reference the new node, completely integrating it into both ends of the chain.

```cpp
template <typename T>
void dllDelete(T findData, DoublyList<T>** head) {
    if (*head == nullptr) return;

    DoublyList<T>* currentNode = *head;

    while (currentNode != nullptr && currentNode->data != findData) {
        currentNode = currentNode->next;
    }

    if (currentNode == nullptr) return;

    if (currentNode->prev == nullptr) {
        *head = currentNode->next;
        if (*head != nullptr) {
            (*head)->prev = nullptr;
        }
    } else {
        currentNode->prev->next = currentNode->next;
        if (currentNode->next != nullptr) {
            currentNode->next->prev = currentNode->prev;
        }
    }

    delete currentNode;
}
```

template void dllDelete(T findData, DoubleList** head) looks for a node in the list that contains data equal to findData and deletes it. A double pointer is needed because removing the head node necessitates updating the head variable of the caller. It moves ahead with currentNode until it locates a match or encounters nullptr; if a match is found, it checks if currentNode->prev == nullptr (indicating it's the head) — if true, it updates *head to bypass it and adjusts the new head's prev to nullptr; if not, it reconnects the adjacent nodes on both sides (currentNode->prev->next and, if not the last node, currentNode->next->prev) to circumvent the deleted node, then releases it using delete.

``
template <typename T>
void dllDeleteList(DoublyList<T>** head) {
    DoublyList<T>* current = *head;
    while (current != nullptr) {
        DoublyList<T>* temp = current;
        current = current->next;
        delete temp;
    }
    *head = nullptr;
}
``
	template void dllDeleteList(DoubleList** head) (the last added version) deallocates every node in the list. It moves ahead with current, storing each node in temp prior to progressing to temp->next and subsequently removing temp — this sequence prevents reading ->next from memory that has been deallocated. It doesn't have to modify prev pointers because each node is being deleted regardless, and ultimately assigns *head = nullptr to prevent the caller's head pointer from referencing freed memory.

Table 3-4. Modified Operations for Doubly Linked Lists

7. Supplementary Activity

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

CODE AND ANALYSIS

HOA3.1_JDC > main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "SUPP3_1_JDC_playlist_LL.h"

int main() {
    CircularList<std::string>* playlist = nullptr;

    cllInsertEnd(std::string("Song#1"), &playlist);
    cllInsertEnd(std::string("Song#2"), &playlist);
    cllInsertEnd(std::string("Song#3"), &playlist);
    cllInsertEnd(std::string("Song#4"), &playlist);

    std::cout << "Initial Playlist:\n";
    cllPlayAll(playlist);
    std::cout << std::endl;

    std::cout << "Adding 'Song#3' to the playlist:\n";
    cllInsertEnd(std::string("Song#3"), &playlist);
    cllPlayAll(playlist);
    std::cout << std::endl;

    std::cout << "Removing 'Song#2' from the playlist:\n";
    cllDeleteNode(std::string("Song#2"), &playlist);
    cllPlayAll(playlist);
    std::cout << std::endl;

    CircularList<std::string>* current = playlist;

    std::cout << "Currently Playing: " << current->data << "\n";
    cllPlayAll(playlist);

    current = cllNext(current);
    std::cout << "Next Song: " << current->data << "\n";

    current = cllNext(current);
    std::cout << "Next Song: " << current->data << "\n";

    current = cllPrevious(playlist, current);
    std::cout << "Previous Song: " << current->data << "\n";
    std::cout << std::endl;

    std::cout << "Removing Playslist:\n";
    cllDeleteList(&playlist);
    cllPlayAll(playlist);

    return 0;
}
```

```cpp
#ifndef SUPP3_1_JDC_PLAYLIST_LL_H
#define SUPP3_1_JDC_PLAYLIST_LL_H

#include <iostream>

template <typename T>
class CircularList {
public:
    T data;
    CircularList<T> *next = nullptr;
};

template <typename T>
CircularList<T>* createNode(T newData) {
    CircularList<T>* newNode = new CircularList<T>;
    newNode->data = newData;
    newNode->next = newNode;
    return newNode;
}

template <typename T>
void cllInsertEnd(T newData, CircularList<T>** head) {
    CircularList<T>* newNode = new CircularList<T>;
    newNode->data = newData;

    if (*head == nullptr) {
        newNode->next = newNode;
        *head = newNode;
        return;
    }

    CircularList<T>* currentNode = *head;
    while (currentNode->next != *head) {
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;
    newNode->next = *head;
}

template <typename T>
void cllDeleteNode(T findData, CircularList<T>** head) {
    if (*head == nullptr) return;

    CircularList<T>* currNode = *head;
    CircularList<T>* prevNode = nullptr;

    do {
        if (currNode->data == findData) break;
        prevNode = currNode;
        currNode = currNode->next;
    } while (currNode != *head);
```

```
    if (currNode->data != findData) {
        std::cout << "Song not found";
        return;
    }

    if (prevNode == nullptr) {
        CircularList<T>* lastNode = *head;
        while (lastNode->next != *head) {
            lastNode = lastNode->next;
        }
        lastNode->next = currNode->next;
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}

template <typename T>
void cllPlayAll(CircularList<T>* head) {
    if (head == nullptr) {
        std::cout << "Playlist is Empty\n";
        return;
    }

    CircularList<T>* currentNode = head;
    do {
        std::cout << currentNode->data;
        currentNode = currentNode->next;
        if (currentNode != head) {
            std::cout << " -> ";
        }
    } while (currentNode != head);
    std::cout << " -> " << head->data << "\n";
}

template <typename T>
CircularList<T>* cllNext(CircularList<T>* currentSong) {
    if (currentSong == nullptr) return nullptr;
    return currentSong->next;
}

template <typename T>
CircularList<T>* cllPrevious(CircularList<T>* head, CircularList<T>* currentSong) {
    if (head == nullptr || currentSong == nullptr) return nullptr;

    CircularList<T>* temp = head;
    while (temp->next != currentSong) {
```

```
    while (temp->next != currentSong) {
        temp = temp->next;
    }
    return temp;
}

template <typename T>
void cllDeleteList(CircularList<T>** head) {
    if (*head == nullptr) return;

    CircularList<T>* current = (*head)->next;
    while (current != *head) {
        CircularList<T>* temp = current;
        current = current->next;
        delete temp;
    }
    delete *head;
    *head = nullptr;
}

#endif
```

This program for a circular doubly/singly linked list simulates a music playlist system by employing a custom generic template class (`CircularList`) for smooth, uninterrupted navigation. By dynamically connecting nodes in a closed loop where the final element refers back to the head, the program removes boundary issues such as hitting the end of a track queue. Essential linked list functions including node generation (`createNode`), appending to the tail (`cllInsertEnd`), node removal (`cllDeleteNode`), and comprehensive memory deallocation (`cllDeleteList`) are adjusted to preserve cycle integrity constantly, ensuring that circular links are intact throughout additions or deletions.

From a structural perspective, the execution demonstrates how linear data structures may be modified for sequential user interfaces featuring cyclic playback. Functions such as `cllNext` utilize direct pointer movement for $O(1)$ forward advancement, whereas `cllPrevious` illustrates the inherent compromises of a singly linked circular structure by manually navigating the ring to find the preceding node. The `main` driver program verifies these mechanics by generating a playlist, adding duplicate entries, deleting certain tracks, moving forward and backward through songs, and securely deallocating all dynamic memory—resulting in a streamlined, memory-efficient structure for queue management.

OUTPUT

```
Initial Playlist:
Song#1 -> Song#2 -> Song#3 -> Song#4 -> Song#1

Adding 'Song#3' to the playlist:
Song#1 -> Song#2 -> Song#3 -> Song#4 -> Song#3 -> Song#1

Removing 'Song#2' from the playlist:
Song#1 -> Song#3 -> Song#4 -> Song#3 -> Song#1

Currently Playing: Song#1
Song#1 -> Song#3 -> Song#4 -> Song#3 -> Song#1
Next Song: Song#3
Next Song: Song#4
Previous Song: Song#3

Removing Playslist:
Playlist is Empty
```

8. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For code debugging, code explanation and clarification, and grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI did not replace my original thoughts, analysis, and authorship. All indicated assistance from AI has been declared, and nothing more from the said assistance has removed my work in the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.